PES-VCS Lab Report

Name: Chatresh Ramasai Gudi
SRN: PES1UG24CS918
Repository: PES1UG24CS918-pes-vcs

Phase 1: Object Storage Foundation

Screenshot 1A — `./test_objects` output (all tests passing)

```
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$ S
```

Screenshot 1B — `find .pes/objects -type f` (sharded directory structure)

```
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$
```

Phase 2: Tree Objects

Screenshot 2A — `./test_tree` output (all tests passing)

```
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B — `xxd` of a raw object (first 20 lines, showing binary format)

```
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$ ./test_objects
find .pes/objects -type f
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | hea
d -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
 | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
bash: syntax error near unexpected token `|'
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$ xxd .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476 | hea
d -20
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$
```

Phase 3: The Index (Staging Area)

Screenshot 3A — `pes init` → `pes add` → `pes status` sequence

```
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$ make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_tree
  untracked:  object.c
  untracked:  phase1_commits.sh
  untracked:  test_sequence.sh
  untracked:  tree.h
  untracked:  index.h
  untracked:  .gitignore
  untracked:  test_objects.c
  untracked:  test_objects
  untracked:  pes.h
  untracked:  pes.c
  untracked:  Makefile
  untracked:  commit.h
  untracked:  commit.c
  untracked:  tree.c
  untracked:  index.c
  untracked:  README.md
  untracked:  test_tree.c
```

Screenshot 3B — `cat .pes/index` (text-format index file)

```
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776402804 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776402804 6 file2.txt
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$
```

Phase 4: Commits and History

Screenshot 4A — `pes log` output (three commits)

```
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776402804 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776402804 6 file2.txt
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$
```

Screenshot 4B — `find .pes -type f | sort` (object store growth)

```
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/51/4d97a7ebd677e409510f743850a06b453f706016c260c4df310ebb28d30ee2
.pes/objects/65/ec20a30abada6c6f0f574f2ab6ccac2e810a49d54c54cbe80ca8c81b10c58f
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/c8/b91ad16ca5bb965c589cf453f3cac7873b579838948b55bf39f14a067c1071
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
.pes/refs/heads/main
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$
```

Screenshot 4C — `cat .pes/refs/heads/main` and `cat .pes/HEAD`

```
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
c8b91ad16ca5bb965c589cf453f3cac7873b579838948b55bf39f14a067c1071
ref: refs/heads/main
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$
```

Final — `make test-integration` (full integration test)

```
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 23a640358de6... Initial commit

Log after first commit:
commit 23a640358de6a3fadee307e5a5e499f20edaf3b508b18cf4771b8c1906aac1ef
Author: Chatresh Ramasai Gudi <PES1UG24CS918>
Date:   1776404424

    Initial commit◆◆


--- Second Commit ---
Committed: 5437663d923a... Update file.txt
```

```
--- Second Commit ---
Committed: 5437663d923a... Update file.txt

--- Third Commit ---
Committed: 8bf7f6a19d90... Add farewell

--- Full History ---
commit 8bf7f6a19d9015c64222693e1ff71be4b5c97c1af7e7b917e2dfd0f024bcc2bb
Author: Chatresh Ramasai Gudi <PES1UG24CS918>
Date:   1776404424

    Add farewell

commit 5437663d923a2118cd7683c52368d63a1296a4913a9b302e03163fda60a78206
Author: Chatresh Ramasai Gudi <PES1UG24CS918>
Date:   1776404424

    Update file.txt

commit 23a640358de6a3fadee307e5a5e499f20edaf3b508b18cf4771b8c1906aac1ef
Author: Chatresh Ramasai Gudi <PES1UG24CS918>
Date:   1776404424

    Initial commitA


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
8bf7f6a19d9015c64222693e1ff71be4b5c97c1af7e7b917e2dfd0f024bcc2bb

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/23/a640358de6a3fadee307e5a5e499f20edaf3b508b18cf4771b8c1906aac1ef
```

```
--- Full History ---
commit 8bf7f6a19d9015c64222693e1ff71be4b5c97c1af7e7b917e2dfd0f024bcc2bb
Author: Chatresh Ramasai Gudi <PES1UG24CS918>
Date:   1776404424

    Add farewell

commit 5437663d923a2118cd7683c52368d63a1296a4913a9b302e03163fda60a78206
Author: Chatresh Ramasai Gudi <PES1UG24CS918>
Date:   1776404424

    Update file.txt

commit 23a640358de6a3fadee307e5a5e499f20edaf3b508b18cf4771b8c1906aac1ef
Author: Chatresh Ramasai Gudi <PES1UG24CS918>
Date:   1776404424

    Initial commitA


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
8bf7f6a19d9015c64222693e1ff71be4b5c97c1af7e7b917e2dfd0f024bcc2bb

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/23/a640358de6a3fadee307e5a5e499f20edaf3b508b18cf4771b8c1906aac1ef
.pes/objects/54/37663d923a2118cd7683c52368d63a1296a4913a9b302e03163fda60a78206
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/8b/f7f6a19d9015c64222693e1ff71be4b5c97c1af7e7b917e2dfd0f024bcc2bb
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
vboxuser@ubuntu:~/Desktop/OS/Orange_U4/PES1UG24CS918-pes-vcs$
```

Phase 5: Branching and Checkout (Analysis)

Q5.1 — How would you implement `pes checkout <branch>`?

A branch in PES-VCS (like Git) is simply a file in `.pes/refs/heads/` containing a commit hash. Implementing `pes checkout <branch>` requires the following steps:

Files that must change in `.pes/`:

1. `.pes/HEAD` must be updated to `ref: refs/heads/<branch>` (pointing to the new branch).
2. If the branch doesn't exist yet (creating a new branch), a new file `.pes/refs/heads/<branch>` must be created with the current commit hash.

Working directory changes:

1. Read the target branch's commit hash from `.pes/refs/heads/<branch>`.
2. Read the commit object to get its tree hash.
3. Walk the tree recursively and restore every file in the tree to its stored blob content.

4. Files present in the working directory but not in the target tree must be deleted (unless they are untracked).

What makes this complex:

- **Three-way merge of working states:** Files can exist in (a) the current HEAD tree, (b) the target branch's tree, and (c) the working directory. Safe checkout must compare all three states.
- **Untracked files:** Files not in either tree must never be deleted — they belong to the user, not the VCS.
- **Dirty working directory:** If a file has unstaged changes and differs between the two branches, checkout must refuse (see Q5.2).
- **Atomicity:** Either all files switch to the new branch state or none do — a crash halfway would leave the repo in a corrupt mixed state.

Q5.2 — How to detect a dirty working directory conflict during checkout?

To detect whether switching branches is safe without re-hashing every file, we can use the index as a fast intermediary:

1. **Load the current index** (staged state).
2. **For each file tracked in the index**, compare its stored `mtime` and `size` metadata against the actual file on disk using `stat()`.
   - If either differs → the file has been **modified but not staged** → "dirty".
3. **For the dirty files found**, check whether the target branch's tree contains a *different* blob hash for the same path.
   - Fetch the target tree from the object store, find the entry matching the file's path.
   - Compare its blob hash against the index entry's hash.
   - If they differ → **conflict**: the file is locally modified AND the branch would overwrite it → refuse checkout with an error.
4. If no conflicts are found, proceed with checkout.

This approach avoids re-reading and hashing file contents entirely — it uses the metadata fast-path (mtime + size) then only looks up object hashes for the subset of files that appear dirty.

Q5.3 — What is "Detached HEAD" and how to recover lost commits?

Detached HEAD means `.pes/HEAD` contains a raw commit hash directly (e.g. `a1b2c3d4...`) instead of a branch reference (e.g. `ref: refs/heads/main`). This happens when you checkout a specific commit rather than a branch name.

What happens if you commit in this state:

- `commit_create` calls `head_update`, which sees HEAD contains a raw hash (not `ref: ...`) and writes the new commit hash directly into HEAD.
- The new commit is correctly created and stored in the object store.
- However, **no branch pointer moves** — the new commit is only reachable from HEAD itself.
- The moment you switch to any branch, HEAD gets overwritten and the new commit becomes **unreachable** — no branch, no ref points to it.

How to recover:

1. Find the "lost" commit hash — it was printed by `pes commit` when it was created. If that output was saved (e.g. in terminal history), use it directly.
2. Alternatively, scan all objects in `.pes/objects/` looking for commit-type objects. Compare timestamps or messages to identify the lost ones.
3. Once the hash is found, create a new branch pointing to it:

   ```
   echo "<lost-commit-hash>" > .pes/refs/heads/recovery
   ```

4. Switch to that branch via `pes checkout recovery` to re-attach the commits to a named branch.

In real Git, `git reflog` maintains a log of every HEAD position change, making recovery trivial.

Phase 6: Garbage Collection (Analysis)

Q6.1 — Algorithm to find and delete unreachable objects

Algorithm (Mark and Sweep):

Mark phase — find all reachable hashes:

1. Start from all branch refs in `.pes/refs/heads/` (the "roots").
2. For each branch, read the commit hash. Add it to a **reachable set** (a hash set / unordered_set for O(1) lookup).
3. Read the commit object:
   - Add its **tree hash** to the reachable set.
   - Recursively walk the tree: for each entry, add blob hashes; for subtrees, recurse and add those tree hashes too.
   - If the commit has a **parent**, add the parent hash and repeat from step 3 for the parent.
4. Continue until all commits in the history chain have been visited.

Sweep phase — delete unreachable objects:

1. Walk every file in `.pes/objects/XX/YYYYYYY...` using `find` or `opendir/readdir`.
2. Reconstruct the full hash from the directory name + filename.
3. If the hash is **not in the reachable set** → delete the file.

Data structure: An **unordered hash set** (e.g. C `uthash`, Python `set`, or a bitset over hash prefixes) gives O(1) average-case membership checks.

Estimation for 100,000 commits, 50 branches:

- Average files per commit: ~10 blobs + 3 trees + 1 commit = ~14 objects per commit.
- Unique objects (with heavy deduplication): ~100,000 × 14 × 0.3 (dedup factor) ≈ **420,000 objects to visit**.
- Reachable set size: ~420,000 hash entries × 32 bytes = ~13 MB in memory — easily manageable.

Q6.2 — Race condition between GC and concurrent commit

The race condition:

Consider this interleaving:

Time	GC process	Commit process
T1	Scans object store, builds reachable set	—
T2	—	Writes new blob object to `.pes/objects/`
T3	—	Writes new tree object to `.pes/objects/`
T4	GC sweep begins	—
T5	GC sees blob — **not yet in any commit** → marks unreachable	—
T6	GC **deletes the blob**	—
T7	—	Writes commit object referencing the now-deleted tree/blob
T8	—	Updates HEAD → commit points to missing object!

The repository is now **corrupt**: the commit object references a blob that no longer exists.

How Git avoids this:

1. **Grace period / "recent objects" exemption:** Git's GC (`git gc`) never deletes objects created within the last 2 weeks (configurable via `gc.pruneExpire`). Since a commit in progress takes milliseconds, any objects written during a concurrent commit are "too recent" to delete.

2. **Lock files:** Git uses `.git/index.lock` and ref locks during write operations. GC can check for active lock files and refuse to run, or pause the sweep.

3. **Two-phase GC with a safety window:** Git first writes a list of "to be pruned" objects, waits for a configurable delay (e.g. 12 hours for `--prune=now` vs default 2 weeks), then deletes. Any objects referenced by a new commit within that window are spared.

The fundamental principle: **GC must only delete objects that have been unreachable for longer than the maximum possible duration of a concurrent write operation.**